

Observing and Controlling Features in Vision-Language-Action Models

Hugo Buurmeijer*, Carmen Amo Alonso*, Aiden Swann[†] and Marco Pavone*[‡]

*Dept. of Aeronautics and Astronautics, Stanford University

[†]Dept. of Mechanical Engineering, Stanford University

[‡]NVIDIA Research

Abstract—Vision-Language-Action Models (VLAs) have shown remarkable progress towards embodied intelligence. While their architecture partially resembles that of Large Language Models (LLMs), VLAs exhibit higher complexity due to their multi-modal inputs/outputs and often hybrid nature of transformer and diffusion heads. This is part of the reason why insights from mechanistic interpretability in LLMs, which explain how the internal model representations relate to their output behavior, do not trivially transfer to VLA counterparts. In this work, we propose to close this gap by introducing and analyzing two main concepts: *feature-observability* and *feature-controllability*. In particular, we first study features that are linearly encoded in representation space, and show how they can be observed by means of a linear classifier. Then, we use a minimal linear intervention grounded in optimal control to accurately place internal representations and steer the VLA’s output towards a desired region. Our results show that targeted, lightweight interventions can reliably steer a robot’s behavior while preserving closed-loop capabilities. We demonstrate on different VLA architectures ($\pi_{0.5}$ and OpenVLA) through simulation experiments that VLAs possess interpretable internal structure amenable to online adaptation without fine-tuning, enabling real-time alignment with user preferences and task requirements.

I. INTRODUCTION

Vision-Language-Action models (VLAs) represent a significant step towards embodied intelligence. By processing multi-modal inputs consisting of images, language instructions, and proprioceptive signals, VLAs combine perception, reasoning, and action generation in a single model. Similar to other disciplines where generative models have unlocked unprecedented degrees of generalization, VLAs enable robots to interpret natural language commands in rich visual contexts and execute complex behaviors, exhibiting improved generalization across tasks and environments [8, 6]. However, VLAs suffer from similar limitations as other generative models: their behavior can be unpredictable, difficult to correct real-time, or misaligned with user preferences and safety requirements [4].

One promising approach to overcoming these limitations is the ability to steer generative models toward desired behaviors while suppressing undesirable ones. In the context of Large Language Models (LLMs), intervening internal model representations to influence outputs has become a prominent research direction, referred to as “activation steering” [18]. Achieving a similar level of control is arguably even more critical for VLAs, where steering directly affects physical behavior in the real world. Furthermore, since VLAs often share architectural components with LLMs, (most notably

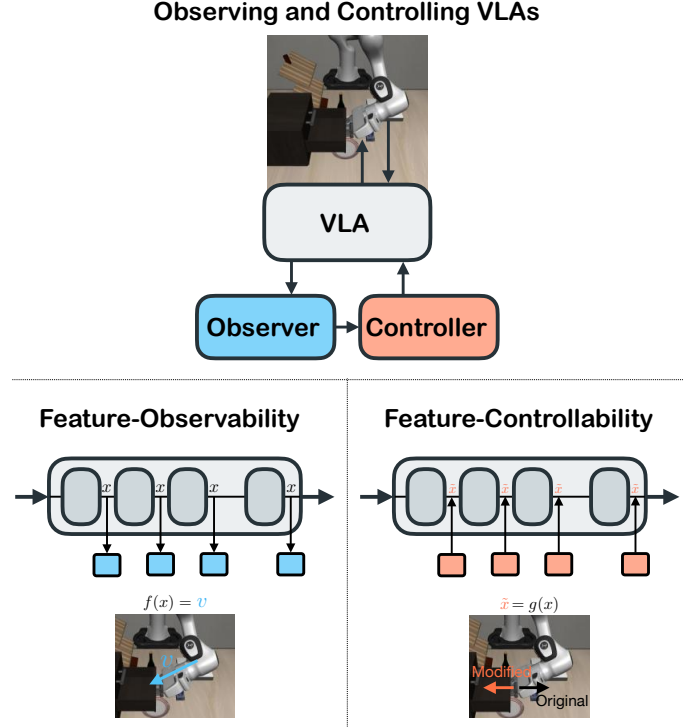


Fig. 1. We present the combination of *feature-observability* and *feature-controllability*: a framework to observe and control robot behavior through accessing and modifying internal VLA representations. A linear observer extracts features from the Transformer’s internal representations (left), while a minimal linear controller steers these representations to align outputs with desired constraints (right), enabling real-time policy steering without fine-tuning.

transformer-based representations [19]), methods developed for LLMs could in theory transfer to VLAs without significant overhead. However, the fact that VLAs operate over multiple modalities, produce continuous action outputs, and must function in closed-loop interaction with the physical world, pose challenges that are not present in LLMs.

To close this gap, several works have explored porting concepts from mechanistic interpretability in LLMs over to the domain of VLAs [5, 13, 12]. For instance, [5] overwrites VLA model activations at inference time to modulate basic motion features such as speed and direction. Overall, the effectiveness of existing methods is still limited. Crucially, while the focus of activation steering methods in LLMs is to control the behavior of the model while preserving its original

merits, such as natural-sounding generations, in the field of VLAs interventions that preserve natural behaviors have not been studied. Hence, achieving reliable and natural control of VLAs' behavior remains an unresolved challenge with critical consequences. Lack of accurate control hinders reliable real-world deployment and motivates the need for methods that can observe and precisely steer VLA behavior without sacrificing their generative flexibility or closed-loop performance.

In this paper, we address this challenge by formalizing concepts that are often implicit in work on steering LLMs, with the goal of making them applicable to VLAs. We introduce a unified framework for analyzing and influencing VLA behavior through their internal representations, grounded in the classical notions of observability and controllability. Within this framework, we propose specific instances of lightweight observer and controller structures that show how output-relevant features can be both identified and manipulated directly in representation space at inference time. Importantly, our approach builds on existing insights from LLM interpretability and steering, with a particular focus on preserving naturalness of the generations. Concretely, our **contributions** are as follows:

- We introduce the concepts of *feature-observability* and *feature-controllability* for generative models, formalizing when behaviorally relevant features are accessible and steerable through internal representations. (Section III)
- We propose a linear observer for transformer layers in VLAs that leverages the linear representation hypothesis [14], enabling efficient extraction of meaningful features. (Section IV-A)
- We design a linear controller that operates on transformer representations using these observations and minimally perturbs the original activations, thereby preserving the naturalness of the model's behavior. (Section IV-B)
- We present an online algorithm that integrates the observer and controller in closed-loop operation without requiring fine-tuning or retraining. (Section IV-C)
- We validate the proposed framework through extensive simulation and real-world experiments across different VLA architectures. (Section V)

II. RELATED WORK

A. Mechanistic Interpretability in VLAs

The intersection of mechanistic interpretability and robotics, specifically VLAs, is significantly underexplored. Nonetheless, a number of recent works have investigated the use of those tools in the context of end-to-end robot control. Notably, it has been shown that steering vectors can be extracted from the feed-forward layers of the Transformer backbone to causally affect model behaviors such as speed and height [5]. More abstract concepts, such as spatial relationships between objects and movement targets, were extracted from VLA models in [12] by means of linear probes.

Beyond interpretability and behavior steering, recent work has explored leveraging similar tools to improve performance

on new tasks. For instance, [10] extract task-specific “text latent” vectors from π_0 's hidden states that can be used to reconstruct demonstrated behaviors or blended to compose novel skills, improving extrapolation performance from 9% to 83% on out-of-distribution tasks. Furthermore, VLA models can be adapted to new tasks by mechanistically identifying and fine-tuning the task relevant attention heads [13].

B. Activation Steering in LLMs

A growing body of work has shown that high-level semantic and behavioral features in LLMs are often linearly represented in their activation spaces, a phenomenon commonly referred to as the *linear separability hypothesis* [14, 15]. It has been widely observed that simple linear classifiers can reliably extract attributes such as sentiment, topic, or personality traits from intermediate transformer layers [3, 16, 2], suggesting that these representations encode disentangled, behaviorally meaningful features. Building on this insight, *activation steering* methods modify internal activations at inference time to influence model outputs without fine-tuning [21, 17]. Techniques such as activation addition [18, 9] have been applied to guide generation toward desired attributes, mitigate harmful behaviors, and improve safety and alignment. Crucially, these methods emphasize preserving the naturalness and coherence of generated text by applying low-rank or minimal perturbations to the model's internal states.

III. PROBLEM STATEMENT

In this section, we present the problem of how the internal representations in VLAs can be directly controlled to align their outputs with user preferences. To that end, we describe the internal structure of two different families of VLA architectures, as captured by OpenVLA [8] and $\pi_{0.5}$ [6]. We then introduce two concepts that enable the formalization of the problem in mathematical terms.

A. Visual-Language-Action Models (VLAs)

The goal is to formalize the study of internal representations in VLAs. Here we present two different types of **VLA architectures**: (a) **transformer-based**, such as OpenVLA [8], and (b) **transformer-flow-matching** hybrids, such as $\pi_{0.5}$ [6].

a) **Transformer-based VLAs**: This architecture consists of a Vision-Language-Model autoregressive transformer architecture, where visual observations (processed through vision encoders such as DINOv2 and SigLIP) and language instructions are encoded into a shared token space, and a language model backbone (e.g., Llama 2) predicts tokenized output actions autoregressively, which are then decoded into robot commands. Examples of this architecture are OpenVLA [8] and RT2 [22].

b) **Transformer-Flow-Matching hybrid VLAs**: This architecture consists of an autoregressive transformer architecture, where a pretrained Vision-Language Model (VLM) processes visual and language inputs, while a separate “action expert” transformer uses conditional flow matching to generate continuous, high-frequency action trajectories.. An example of

this architecture is the family of π models, such as π_0 [1] and $\pi_{0.5}$ [6].

We note that since the focus of this work is to port existing tools and insights from LLMs into VLAs, we focus on the transformer architectural component part present in VLA architectures. To that end, the remainder of the paper is focus on *internal representations inside the transformer block*. Yet, we show that leveraging these internals directly relates to relevant action features of the VLA output.

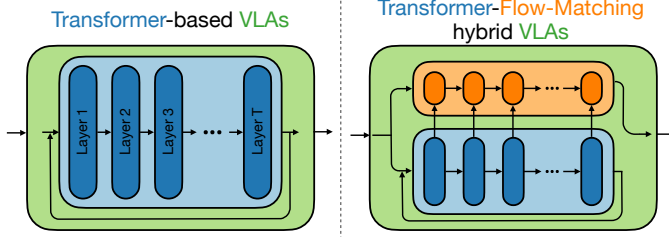


Fig. 2. Schematic representation of two VLA architectures. On the left, a Transformer-based VLA features an autoregressive Transformer architecture. On the right, a Transformer-Flow-Matching hybrid VLA is composed of a Transformer architecture and a Flow Matching block (referred to as the “action expert”), where the Flow Matching layers attend to their paired counterparts in the Transformer.

We formalize the Transformer architecture [19] as a sequence of feedforward transformations (layers) acting on a latent representation of an input sequence, denoted as s . The model first maps this input sequence into a continuous representation via an embedding map E , producing an initial hidden state

$$x_0 = E(s), \quad x_0 \in \mathbb{R}^d. \quad (1)$$

The model then applies a sequence of T Transformer layers. Each layer $L_\ell : \mathbb{R}^d \rightarrow \mathbb{R}^d$ transforms the hidden state from the previous layer, yielding the recurrence

$$x_{\ell+1} = L_{\ell+1}(x_\ell), \quad \ell = 0, \dots, T-1, \quad (2)$$

where $x_\ell \in \mathbb{R}^d$ is the latent representation after the ℓ -th layer.

We work under an assumption on the VLA representation space that is analogous to common assumptions made for LLM representations. Namely, we assume that the latent representation x_ℓ encodes information about a set of important features of the robot’s behavior. That is, there exists an (unknown) mapping from the hidden state to these features, reflecting the common assumption in LLMs that different dimensions or subspaces of the representation capture distinct aspects of the input. Examples of features includes, for instance, the actions and states of the robot. We denote features as $\zeta \in \mathbb{R}^n$.

Since the Transformer architecture is embedded into a VLA model, the action output a is computed using the internal representations, i.e., $a = \phi(x_1, \dots, x_T)$, where the map ϕ depends on the choice of architecture: for the transformer-based architecture ϕ depends only on the final layer representation, x_T , while for the transformer-flow-matching hybrid architecture ϕ includes the flow matching process that can be conditioned on intermediate layer representations $x_1, \dots, x_T$.

B. Observability and Controllability of Features in VLAs

In this paper, we explore how the structure of the representation space in VLAs can be leveraged to (i) observe features, such as states or actions, and (ii) steer features into a desired region by controlling their representations. To that end, we introduce the concepts of *feature-observability* and *feature-controllability*.

Definition 1 (Feature-Observability). *Given the activation $x_\ell \in \mathbb{R}^d$ in layer ℓ of a Transformer architecture, a feature $\zeta \in \mathbb{R}^n$ is observable at layer ℓ if there exists a map (referred to as the observer) $f_\ell : \mathbb{R}^d \rightarrow \mathbb{R}^n$; $x \mapsto \zeta$ such that $f_\ell(x_\ell) = \zeta$.*

Definition 2 (Feature-Controllability). *Given a desired set $\mathcal{D} \in \mathbb{R}^d$, a feature $\zeta \in \mathbb{R}^n$ is controllable at layer ℓ of a Transformer architecture if there exists a map (referred to as the controller) $g_\ell : \mathbb{R}^d \rightarrow \mathbb{R}^d$; $x \mapsto \tilde{x}$ such that when the modified intervention $\tilde{x}_\ell \in \mathbb{R}^d$ is propagated through layers $\ell, \dots, T$ via equations (1) and (2), it leads to $\zeta \in \mathcal{D}$.*

Given the definitions above, the open question is how to design observer, $f(\cdot)$, and controller maps, $g(\cdot)$, that enable feature-observability and feature-controllability for relevant features in representation space, and how to use them effectively during VLA generations. In what follows, we propose that linear maps are sufficient to effectively observe features and align VLA’s outputs towards user preferences at inference.

IV. APPROACH

In this section, we formalize our approach through the two central concepts: *feature-observability* and *feature-controllability* previously introduced. Specifically, we propose a linear observer structure that allows for information about interpretable features in the model to be extracted from its internal state (feature observer); and a linear controller structure that steers features in the model’s internal representations through interventions to its internal state (feature controller). We then show how these proposals enable effective and efficient feature-observability and -controllability at inference.

A. Designing the Feature Observer

We design the observer map at layer ℓ , $f_\ell(\cdot)$, to be a linear function. This is, given a representation $x_\ell \in \mathbb{R}^d$, a feature of interest $\zeta \in \mathbb{R}^n$ can be found as $\zeta = f_\ell(x_\ell)$ with

$$f_\ell(x) := W_\ell x + b_\ell, \quad (3)$$

where $W_\ell \in \mathbb{R}^{d \times n}$ and $b_\ell \in \mathbb{R}^n$ are learned parameters.

In this paper, we restrict our features of interest to robot states and actions, and we defer studying more abstract semantic features such as object affordances, relational predicates, and task-level goals to future work. This choice is well-motivated: robot states and actions constitute the fundamental observation and control space for robotic systems, are directly measurable and actuatable, and generalize naturally across diverse tasks. Formally, we define the robot state space as $s = (x, y, z, \phi, \theta, \psi, g)$, where $(x, y, z) \in \mathbb{R}^3$ represent Cartesian position, $(\phi, \theta, \psi) \in [0, 2\pi)^3$ encode roll, pitch, and

yaw orientations, and $g \in [0, 1]$ represents normalized gripper aperture. Robot actions $a = \Delta s \in \mathcal{A}$ correspond to relative displacements in this state space.

The weights $W_\ell \in \mathbb{R}^{d \times n}$ and the bias $b_\ell \in \mathbb{R}^n$ are learned from data. In particular, we leverage pairs of inputs (prompt sequence and set of images, denoted as s) to an feature of interest (denoted as $\zeta \in \mathbb{R}^d$). Given a set of input-feature pairs $\{s^{(i)}, \zeta^{(i)}\}_{i=1}^N$, a set of activation-feature pairs $\{x_\ell^{(i)}, \zeta^{(i)}\}_{i=1}^N$ can be collected at each layer ℓ by simply propagating each input sequence $s^{(i)}$ up until layer ℓ through the transformer architecture via equations (1) and (2), whose output is $x_\ell^{(i)}$. Then, W_ℓ and b_ℓ are the minimizers of the cross-entropy loss over the dataset $\{x_\ell^{(i)}, \zeta^{(i)}\}_{i=1}^N$ for each layer ℓ :

$$W_\ell, b_\ell = \underset{W_\ell, b_\ell}{\operatorname{argmin}} - \sum_{i=1}^N \left[\zeta^{(i)} \log(W_\ell^\top x_\ell^{(i)} + b_\ell) + (1 - \zeta^{(i)}) \log(1 - (W_\ell^\top x_\ell^{(i)} + b_\ell)) \right]. \quad (4)$$

We note that the learning task is a regression task since we are considering continuous values for the feature $\zeta \in \mathbb{R}^n$. Algorithm 1 summarizes the steps to compute the weights W_ℓ and biases b_ℓ of the observer at layer ℓ , $f_\ell(\cdot)$.

Algorithm 1 Learning linear observer f_ℓ (Eq. (3)) at layer ℓ

```

1: Input: Labeled dataset  $\{(s^{(i)}, \zeta^{(i)})\}_{i=1}^N$ 
2: Output:  $W_\ell \in \mathbb{R}^{d \times n}$  and  $b_\ell \in \mathbb{R}^n$ 
3: for  $i = 0, \dots, N$  do       $\triangleright$  In parallel for all  $i = 1, \dots, N$ 
4:    $x_0^{(i)} \leftarrow \text{Eq. (1)}$        $\triangleright$  Compute initial embedding
5:   for  $t = 0, \dots, \ell - 1$  do   $\triangleright$  Sequentially until layer  $\ell$ 
6:      $x_{t+1}^{(i)} \leftarrow \text{Eq. (2)}$   $\triangleright$  Compute layer  $t$  representation
7:   end for
8: end for
9:  $(W_\ell, b_\ell) \leftarrow \text{Eq. (4)}$    $\triangleright$  Train linear observer at layer  $\ell$ 
10: return  $W_\ell, b_\ell$ 

```

The choice of a linear map is motivated by the linear separability hypothesis, widely explored in LLMs [14]. We note that the presented method holds for generalized versions of the linear observer, such as $f_\ell(x) = \nu(W_\ell x + b_\ell)$, where ν is some known monotonic nonlinearity of choice.

Remark 1. We note that finding an observer f_ℓ for a given layer ℓ via Algorithm 1 does not guarantee robustness of such observer. This is, in theory, an epsilon perturbation to internal representation x_ℓ could lead to an arbitrarily large change in the estimated feature ζ . For this reason, after learning the observer f_ℓ via Algorithm 1, we empirically verify that given $\epsilon > 0$, then $\|f_\ell(x + \epsilon) - f_\ell(x)\| < \delta$ for some $\delta > 0$.

B. Designing the Feature Controller

We design the controller map at layer ℓ , $g_\ell(\cdot)$, to be a linear intervention on the state. This is, given a representation $x_\ell \in \mathbb{R}^d$, the modified representation is computed as $\tilde{x}_\ell = g_\ell(x_\ell)$ with

$$g_\ell(x) := x + u_\ell, \quad (5)$$

where u_ℓ is an additive perturbation to the representation. Similar to prior work in LLMs [3], we propose to compute u_ℓ as the solution to the optimization problem:

$$u_\ell = \underset{u \in \mathbb{R}^d}{\operatorname{argmin}} \|u\|_2^2 \quad (6a)$$

$$\text{s.t.} \quad f_\ell(x_\ell + u) \in \mathcal{D}, \quad (6b)$$

where $\mathcal{D} \subset \mathbb{R}^n$ is the desired set of feature values, and $f_\ell(\cdot)$ is the observer map at layer ℓ . In words, u_ℓ is the minimal additive intervention such that the resulting modified representation $x_\ell + u_\ell$ lies in the preimage of the desired feature set under the observer. Intuitively, u_ℓ is the smallest control input in representation space that steers the observed features into a target region.

In general, the constraint in (6b) may be nonconvex, depending on the form of f_ℓ and the geometry of $\mathcal{D}$. To obtain a tractable solution, we use a linear observer as per equation (3), and we assume that $\mathcal{D} = [\zeta_{\min}, \zeta_{\max}] \subset \mathbb{R}$, i.e., some lower and upper bound for one-dimensional ζ values. As shown in [3] (Theorem 4.1), under these assumptions the solution to 6 can be computed in closed form as:

$$u_\ell = (\zeta_{\max} - \zeta_\ell) \frac{W_\ell}{\|W_\ell\|_2^2}, \quad \text{if } \zeta_\ell > \zeta_{\max}, \quad (7a)$$

$$u_\ell = (\zeta_{\min} - \zeta_\ell) \frac{W_\ell}{\|W_\ell\|_2^2}, \quad \text{if } \zeta_\ell < \zeta_{\min}, \quad (7b)$$

$$u_\ell = 0, \quad \text{otherwise}, \quad (7c)$$

where $\zeta_\ell = f(x_\ell) = W_\ell^\top x_\ell + b_\ell$ is the observation at layer ℓ .

We note that the proposed controller leverages the observer. Although feature-observability and feature-controllability are independent properties, i.e. a feature can be observable but not controllable, and vice-versa (as is standard in control theory [7]), more effective control can be achieved when a feature is both observable and controllable. In the LLM literature, while some activation steering proposals inject additive interventions into internal representations without explicit observers [18], their performance is surpassed by those which rely on past observations using labeled data [16, 3]. For this reason, the proposed feature controller relies on observations of the feature. While the generality of this method apply to any observable feature, in this work we restrict ourselves to robot actions, and defer the study of more abstract semantic features to future work.

C. Observing and Controlling Features in VLAs

We illustrate how to leverage the proposed feature observer and feature controller in practice to align VLA outputs with user preferences. Given

- a *feature observer*, $f_\ell(\cdot)$, with linear structure as defined in equation (3), where linear parameters W_ℓ and b_ℓ are computed before inference time via Algorithm 1; and
- a *feature controller*, $g_\ell(\cdot)$, with linear structure as defined in equation (5), where the linear parameter u_ℓ is computed by evaluating the representation x_ℓ according to equation 7,

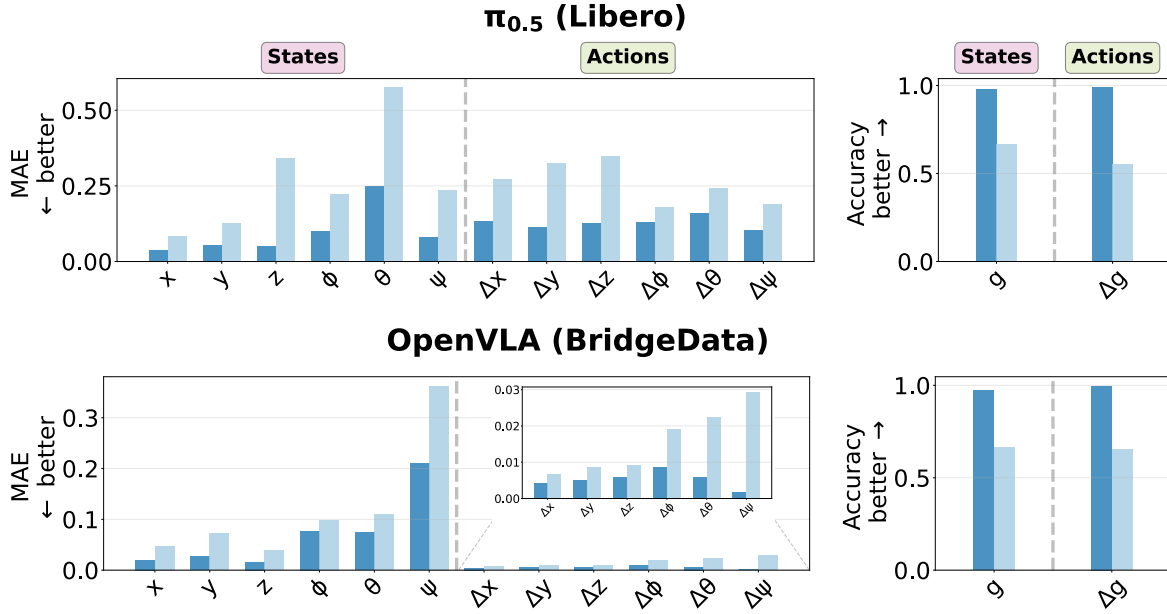


Fig. 3. On the top, results from training a linear classifier via Algorithm 1 on the $\pi_{0.5}$ transformer layers on test data from Libero dataset. Left: the maximum absolute error (MAE) of the **trained classifier** compared with the MAE from **mean training prediction**, used as a baseline. Right: accuracy of the **trained classifier** compared with accuracy from **majority class prediction**, used as a baseline. On the bottom, identical representation, using OpenVLA model on the BridgeData V2 dataset. In all cases, only results for the best performant layer are displayed.

the goal is to integrate them into the inference time computations at layer ℓ to achieve precise representation control and downstream alignment of the VLA output.

Let us consider two set of layers: (i) $\mathcal{L}_O \subseteq [0, \dots, T]$ where feature-observability is desired, and (ii) $\mathcal{L}_C \subseteq [0, \dots, T]$ where feature-controllability is desired. We note that, given that the proposed controller requires observations, $\mathcal{L}_C \subseteq \mathcal{L}_O$. The inference-time computation of a Transformer architecture’s forward pass equipped with observer and controller for feature ζ is described by Algorithm 2.

Algorithm 2 Inference forward-pass in a VLA-embedded Transformer architecture with ζ -observer and ζ -controller

```

1: Input:  $s, \mathcal{L}_O, \mathcal{L}_C, W_{\ell \in \mathcal{L}_O}, b_{\ell \in \mathcal{L}_O}, \zeta_{\min}, \zeta_{\max}$ 
2: Output:  $x_1, \dots, x_T$ 
3:  $x_0 \leftarrow \text{Eq. (1)}$   $\triangleright$  Compute initial embedding
4: for  $\ell = 1, \dots, T$  do  $\triangleright$  Forward-pass through layers
5:    $x_\ell \leftarrow \text{Eq. (2)}$   $\triangleright$  Compute layer  $\ell$  representation
6:   if  $\ell \in \mathcal{L}_O$  then  $\triangleright$  Feature-observability
7:      $\zeta_\ell \leftarrow \text{Eq. (3)}$   $\triangleright$  Compute observation
8:     if  $\ell \in \mathcal{L}_C$  then  $\triangleright$  Feature-controllability
9:        $u_\ell \leftarrow \text{Eq. (7)}$   $\triangleright$  Compute control intervention
10:       $x_\ell \leftarrow \text{Eq. (5)}$   $\triangleright$  Apply control intervention
11:   end if
12: end if
13: end for
14: return  $x_1, \dots, x_T$ 

```

The forward-pass presented in Algorithm 2 only differs from a standard un-observed and un-intervened Transformer forward-pass in steps 6-12. A crucial advantage of choosing

a linear observer and a linear closed-form controller is that computations in step 7, 8 and 9 introduce minimal overhead, which results in a negligible increase in runtime while equipping the model with effective steering capabilities.

Remark 2. It is important to note that VLAs operate as closed-loop systems, while LLM generations are open-loop. Specifically, VLA outputs directly impact the environment in which the VLA operates, which is then taken as an input to compute the next action. Yet, despite this fundamental difference between VLAs and LLMs, the concepts of observability and controllability of features carries over from LLMs to VLAs as long as the closed-loop operation does not cause the input to the VLA to be out-of-distribution with respect to the probe training data. The effectiveness of this method in closed-loop regimes is demonstrated in the next section.

V. RESULTS

In this section, we present the results of exploring the concepts of feature-observability and feature-controllability in two frontier VLA models: OpenVLA and $\pi_{0.5}$. We first show that actions and states can be observed in representation space using a linear observer, and these observations are robust to perturbations. We then study how minimal control interventions to the representation space lead to fine-grained steering of the VLA output and better alignment with user preferences and safety requirements.

A. Feature-Observability

The goal is to asses (i) whether the state and actions of the robot can be observed with a linear classifier as defined in equation (3); and (ii) whether these observations are robust,

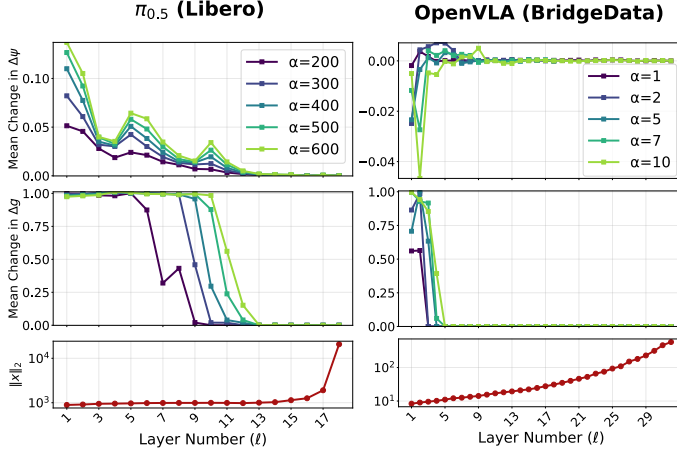


Fig. 4. Effect of linear interventions applied to the representation space of $\pi_{0.5}$ at different layers, using episodes from Libero dataset. On the top, mean change in the delta yaw action, averaged across episodes, as a function of layer where the perturbation (linear offset added to the representation) is applied. The middle similarly shows the mean change in delta gripper action. The perturbation is added to the representation at a single layer ℓ . Each curve corresponds to a different strength α of the perturbation. In the bottom, the L_2 -norm of the representation vector as a function of layer depth is included, showing that representation magnitude increases with depth, which explains the diminishing effect of a fixed perturbation in deeper layers.

i.e., given $\epsilon > 0$, then $\|f_\ell(x + \epsilon) - f_\ell(x)\| < \delta$ for some $\delta > 0$. We perform these experiments by testing $\pi_{0.5}$ [6] on the Libero dataset [11], and OpenVLA [8] on the BridgeData V2 dataset [20].

1) *Actions and States are Linearly Observable in Representation Space:* To learn the linear classifier $f(\cdot)$ in equation (3), we train a probe via Algorithm 1 using annotated data from each of the datasets. For the Cartesian positions and the orientations, a regression probe was used with continuous labels for both the states and actions. For the gripper state and action, a binary probe was used with binarized labels. Probes were trained at every layer $\ell \in [1, \dots, T]$; Figure 3 shows the best performing probe across layers for each state and action.

2) *Action’s and States’ Observations are Robust to Linear Interventions in Representation Space:* To test the robustness of the observations, we add a linear perturbation to the internal representation at layer ℓ , i.e., $x_\ell + \alpha$, where α is the perturbation strength. In each generation, only layer ℓ is perturbed. We do this at each layer individually, and study the mean change in the observed feature (actions) across different episodes. As seen in Figure 4, an increase in α leads to a smooth increase in the mean change from the action for $\pi_{0.5}$. However, inspecting the same result for OpenVLA, we see that in particular the delta yaw action is not robust. The delta gripper action does show an ordering, albeit not as clean as for $\pi_{0.5}$. This result shows that most observations are robust to changes in the representation, and that pushing the representation in the direction perpendicular to the linear classifier leads to a modification of the output feature. We use this fact to influence outputs via the representation space in the

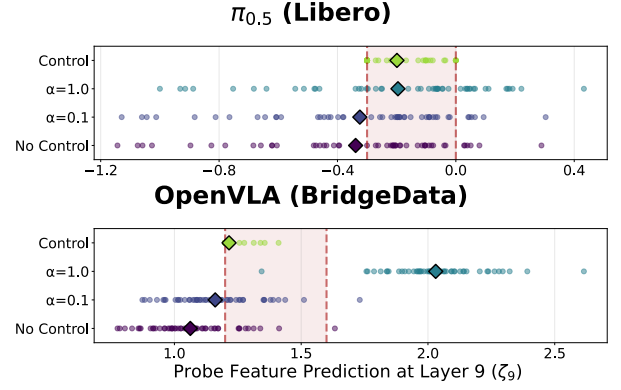


Fig. 5. Visualization of the representation projected into the classifier image space under different interventions of $\pi_{0.5}$ ’s and OpenVLA’s transformer layer 9. The proposed minimal controller from equation (7), observed via equation (3), constrains the image of the intervened representation to lie within the target bounds $[\zeta_{\min}, \zeta_{\max}]$, while other interventions as well as un-intervened representations fall outside.

next subsection. An important observation is that, in general, perturbations to the representation is more effective in earlier layers, and the effect decreases as depth increases. The reason for this is that the L_2 -norm of the representation also increases with depth. As a result, the effect of α becomes smaller as the magnitude of the representation increases. This also explains the difference in absolute values of the perturbations (α) for $\pi_{0.5}$ and OpenVLA to inflict a change in action.

B. Feature-Controllability

The goal is to assess (i) whether the internal representations in the transformer architecture of the VLA can be controlled with a linear intervention as defined in equation (5); and (ii) whether these interventions result in accurate steering of relevant robotic features, such as actions. We perform these experiments by testing $\pi_{0.5}$ [6] on the Libero dataset [11], and OpenVLA [8] on the BridgeData V2 dataset [20].

1) *Optimal Control Interventions Constrain Actions to a Desired Region of Representation Space:* To test the effectiveness of the minimal controller proposed in equation (7), we observe (via the observer in equation (3)) where the representation lies in the classifier image. In particular, we observe that representations appear to be spread out when un-intervened, and also when perturbed with a fixed-size vector αW_ℓ . However, the proposed intervention in equation (7) *guarantees* that the image of the intervened representation lies within the desired bounds $[\zeta_{\min}, \zeta_{\max}]$, since it is a hard constraint in optimization (6). This is also shown through empirical observations in Figure 5.

2) *Optimal Control Interventions in Representation Space allow for Fine-Grained Steering of Actions:* Lastly, we compare how the proposed control technique performs in terms of closed-loop success rate, while satisfying an imposed constraint. All the experiments are based on the Libero simulator [11], a standardized benchmark for evaluating vision-language-action models on robotic manipulation tasks. Specifically, we evaluate on ten tasks from the spatial task suite, and

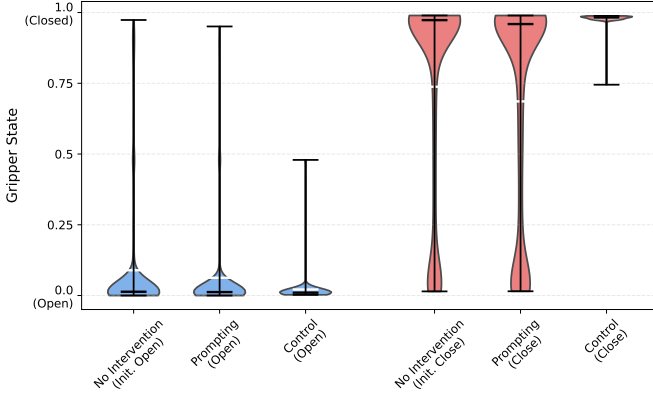


Fig. 6. Violin plots of normalized gripper state during the constraint window, comparing no intervention, prompting, and control under open-initialized (blue) and closed-initialized (red) conditions. Values are normalized to [0,1] where 0 denotes open and 1 denotes closed; means (white) and medians (black) are shown.

in each experiment we perform ten rollouts per task for each method. All inference is done using a single NVIDIA 5090 GPU.

We study three features that we would like to control: gripper state, end-effector height and end-effector speed. For all features we analyze both the absolute steering performance, and the trade off between constraint satisfaction and closed-loop success rate. While previous work has primarily focused on constraint satisfaction or steerability in isolation [5], we strongly argue that steering performance is only meaningful when considered in conjunction with closed-loop task success rate—analogous to how language model steering methods must preserve response naturalness and coherence.

Figure 6 shows strong steerability of gripper states with the method presented in this paper for both “open” and “closed” states. We compare this scenario with the cases of no intervention and prompting, where we provide a favorable initial condition (open gripper if constraint requires “open”, and closed otherwise). While our method achieves near perfect constraint satisfaction, it also maintains a high success rate of above 90%.

Figure 7 similarly shows that the height of the robot’s end-effector can be steered effectively. In particular, when we constrain the end-effector’s height with respect to the robot’s initial condition, we observe that the proposed approach achieves near perfect constraint satisfaction, as shown in Figure 8. Given that the constrained task is strictly harder than its unconstrained counterpart, we observe a (modest) drop in success rate for the control method. We further hypothesize that, for a sufficiently robust base model with good recovery behaviors, this reduction in success rate could be eliminated.

Finally, we constrain the speed of the robot. This feature is not a direct output of the model, but rather a derived quantity computed as $v = \frac{\|\Delta x, \Delta y, \Delta z\|}{dt}$. As shown in Figure 9, we can reliably cause the robot to slow down, but less accurately

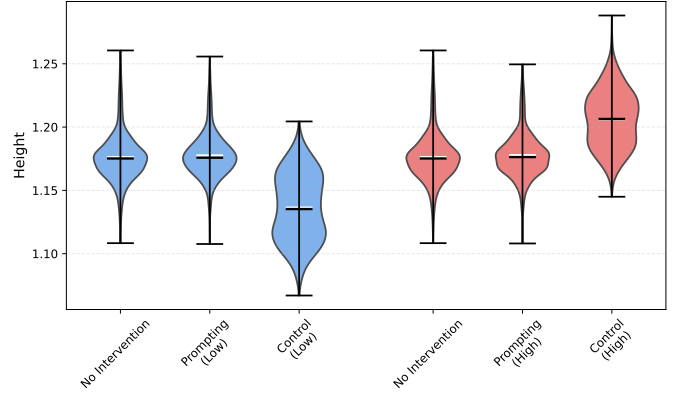


Fig. 7. Violin plots of normalized end-effector height over the first 15 constrained steps in closed-loop runs, comparing no intervention, prompting, and control under low (blue) and high (red) height conditions. Means (white) and medians (black) are overlaid; violins show full distributions across episodes.

cause the robot to speed up. This could be attributed to the lack of training data in the fast speed regime, and is consistent with the results reported in [5]. The same observation can be derived from Figure 10, which further shows that success rates are maintained almost perfectly when controlling speed.

It is important to highlight that one of the main challenges of controlling VLAs is that they operate in closed-loop, as opposed to their LLM counterparts. While LLMs go through a sequence generation without interacting with the physical world, VLAs actions have a direct effect on the physical environment in which they operate, which in turn influences the next input that they receive. It is therefore remarkable that techniques designed to control a forward pass for open-loop generations carry over to closed-loop behavior observed in robots, as illustrated by these results.

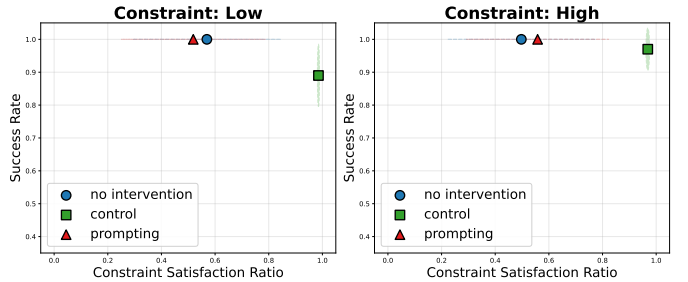


Fig. 8. Comparison of the proposed technique against not intervening and prompting in terms of (closed-loop) Libero task success rate and constraint satisfaction. Two constraints are included: constraining the trajectory to stay below the initial condition (left) and constraining the trajectory to remain above the initial condition (right). Both constraints are set at the start of the trajectory for 15 time steps to allow for task success. The ellipses represent the standard deviations in both directions, which are computed across tasks and episodes.

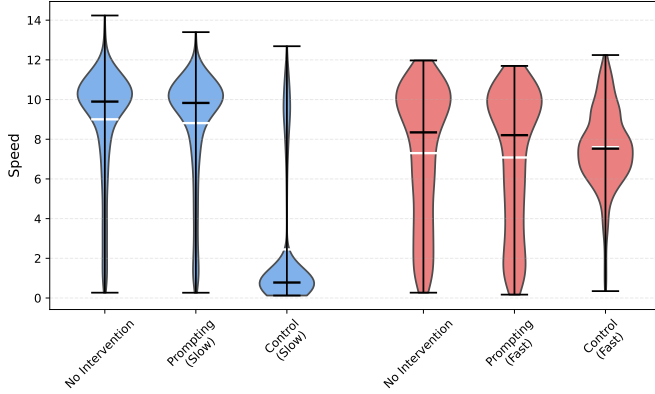


Fig. 9. Violin plots of normalized speed over the first 25 constrained steps in closed-loop runs, comparing no intervention, prompting, and control under low (blue) and high (red) height conditions. Means (white) and medians (black) are overlaid; violins show full distributions across episodes.

VI. CONCLUSION

This paper introduced a principled framework for understanding and controlling Vision-Language-Action models through their internal representations. By formalizing the concepts of *feature-observability* and *feature-controllability*, we established a foundation for interpreting and steering VLA behavior that bridges mechanistic interpretability insights from Large Language Models with the unique challenges of embodied AI systems. Our main contributions demonstrate that: (1) behaviorally relevant features such as robot actions and states are *linearly encoded* in VLA transformer representations and can be reliably extracted using lightweight linear observers; (2) these representations can be precisely controlled through minimal interventions; and (3) this observer-controller framework enables real-time alignment of VLA outputs with user preferences without requiring fine-tuning or retraining.

Through experiments on two state-of-the-art VLA architectures ($\pi_{0.5}$ and OpenVLA) across multiple robotic manipulation datasets (Libero and BridgeData V2), we validated that targeted linear interventions in representation space achieve fine-grained steering of robot behavior while preserving the naturalness and closed-loop capabilities of the original model. Importantly, our approach introduces negligible computational overhead, making it practical for real-time robotic applications.

Limitations and Future Work. While our framework successfully demonstrates feature observability and controllability for robot states and actions, several directions warrant further investigation. First, our current approach requires labeled data to train linear observers, which may not always be available in large-scale robotics datasets. Future work could explore self-supervised or unsupervised methods for feature discovery, such as leveraging Sparse Autoencoders (SAEs) to identify interpretable features without explicit labels. Second, we focused primarily on the transformer components of VLA architectures; extending our framework to the diffusion or flow-matching heads would enable end-to-

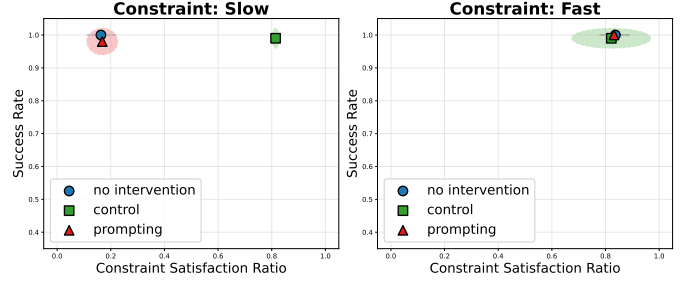


Fig. 10. Comparison of the proposed technique against not intervening and prompting in terms of (closed-loop) Libero task success rate and constraint satisfaction. Two constraints are included: constraining the trajectory to stay below the median speed over training data (left) and constraining the trajectory to remain above the median speed over training data (right). Both constraints are set at the start of the trajectory for 25 time steps to allow for task success. The ellipses represent the standard deviations in both directions, which are computed across tasks and episodes.

end interpretability and control across hybrid architectures. Third, while we demonstrated steering of low-level features, exploring the observability and controllability of higher-level semantic features, such as task goals, object affordances, or spatial relationships, remains an important open question. Finally, establishing principled safety guarantees and bounds on the effects of representation-space interventions would be crucial for deploying these techniques in safety-critical robotic applications.

By demonstrating that VLAs possess interpretable internal structure amenable to lightweight online adaptation, this work takes a step toward making embodied AI systems more transparent, controllable, and aligned with human intent; critical requirements for the reliable deployment of robots in real-world environments.

REFERENCES

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2026. URL <https://arxiv.org/abs/2410.24164>.
- [2] Runjin Chen, Andy Arditi, Henry Sleight, Owain Evans, and Jack Lindsey. Persona vectors: Monitoring and controlling character traits in language models. *arXiv preprint arXiv:2507.21509*, 2025.
- [3] Emily Cheng and Carmen Amo Alonso. Linearly controlled language generation with performative guarantees. *arXiv preprint arXiv:2405.15454*, 2024.
- [4] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness

- analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [5] Bear Häon, Kaylene Stocking, Ian Chuang, and Claire Tomlin. Mechanistic interpretability for steering vision-language-action models. *arXiv preprint arXiv:2509.00328*, 2025.
 - [6] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
 - [7] R.E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82 (1):35–45, 1960.
 - [8] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
 - [9] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. *Advances in Neural Information Processing Systems*, 36: 41451–41530, 2023.
 - [10] Quanyi Li. Task reconstruction and extrapolation for π_0 using text latent. *arXiv preprint arXiv:2505.03500*, 2025.
 - [11] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
 - [12] Hong Lu, Hengxu Li, Prithviraj Singh Shahani, Stephanie Herbers, and Matthias Scheutz. Probing a vision-language-action model for symbolic states and integration into a cognitive architecture. *arXiv preprint arXiv:2502.04558*, 2025.
 - [13] Chancharik Mitra, Yusen Luo, Raj Saravanan, Dantong Niu, Anirudh Pai, Jesse Thomason, Trevor Darrell, Abrar Anwar, Deva Ramanan, and Roei Herzig. Mechanistic finetuning of vision-language-action models via few-shot demonstrations. *arXiv preprint arXiv:2511.22697*, 2025.
 - [14] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *International Conference on Machine Learning*, pages 39643–39666. PMLR, 2024.
 - [15] Kiho Park, Yo Joong Choe, Yibo Jiang, and Victor Veitch. The geometry of categorical and hierarchical concepts in large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [16] Pau Rodriguez, Arno Blaas, Michal Klein, Luca Zappella, Nicholas Apostoloff, Xavier Suau, et al. Controlling language and diffusion models by transporting activations. In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [17] Jiuding Sun, Sidharth Baskaran, Zhengxuan Wu, Michael Sklar, Christopher Potts, and Atticus Geiger. Hypersteer: Activation steering at scale with hypernetworks. *arXiv preprint arXiv:2506.03292*, 2025.
 - [18] Alexander Matt Turner, Lisa Thiergart, David Udell, Gavin Leech, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *CoRR*, 2023.
 - [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
 - [20] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023.
 - [21] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D Manning, and Christopher Potts. Reft: Representation finetuning for language models. *Advances in Neural Information Processing Systems*, 37:63908–63962, 2024.
 - [22] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.